

JDBC

Java 애플리케이션이 표준 java.sql 인터페이스를 통해 데이터베이스 드라이버와 통신하고 SQL 실행 결과를 처리하는 'API'

JDBC: Java DataBase Connectivity

7단계 흐름 : Library import -> Driver load -> Connection 연결 -> 명령객체(Statement) 생성 -> SQL 전송 및 실행 -> 결과 받기 (ResultSet, Integer) -> Resource 해제 및 반납

단계	주요 객체·메서드	설명
드라이버 준비	Connector/J, <code>DriverManager</code>	데이터베이스 벤더 드라이버를 프로젝트에 등록
연결 생성	<code>getConnection()</code>	URL, 계정, 비밀번호로 DB 세션 생성
명령 객체 생성	<code>createStatement()</code>	SQL 입력과 실행을 담당할 객체 statement 생성
SQL 실행	<code>executeQuery()</code> , <code>executeUpdate()</code>	조회 또는 변경 SQL 실행
결과 처리	<code>ResultSet</code> , <code>int</code>	조회 행 또는 영향받은 행 수 처리
자원 해제	<code>close()</code>	결과, 명령, 연결 자원을 역순으로 반납

실습

JDBC 실습 - SELECT

Statement vs PreparedStatement (vs CallableStatement)

Statement : sql문을 담아 서버(연결된 db 세션)에서 실행해주는 객체 다만 SQL 인젝션 위험이 크다!

SQL injection: 사용자가 입력한 데이터가 SQL 쿼리문(코드)의 일부로 해석되어 데이터베이스가 비정상적인 동작을 수행하게 만드는 보안상의 취약점

```
// 사용자가 입력한 ID: "admin"
// 사용자가 입력한 PW: "" OR '1'='1"
String query = "SELECT * FROM users WHERE id = '" + userId + "' AND pw = '" + userPw + "'";
```

PreparedStatement: 값 위치를 ?로 지정하고, set~() 메서드로 값을 바인딩해준다. 기존 stmt와 다르게 구조와 값을 분리해서 사용자가 악의적으로 넣은 데이터가 sql 쿼리문으로 해석되지 않도록 해준다!

```
String sql = "SELECT id, name, phone FROM user WHERE id = ?";
PreparedStatement pstmt = con.prepareStatement(sql);

pstmt.setInt(1, userId); // 첫번째 ? 자리에 userId 값 바인딩
ResultSet rs = pstmt.executeQuery(); //sql 실행 후 rs에 반환
```

CallableStatement: **함수/프로시저** 호출에 쓰인다.

다시 돌아와서.. SELECT를 JDBC로 실행하기

1) 전체 출력

```
public static void selectTest01(){
    Connection conn=null;
    Statement stmt =null;
    ResultSet rs=null;

    try {
        conn= JdbcTemplate.getConnection();
        stmt=conn.createStatement();
        rs=stmt.executeQuery("SELECT id, name, phone FROM user");

        while(rs.next()){ //커서 이동
            int id=rs.getInt(1); //index로 받기 - 숫자
            String name= rs.getString("name"); //label로 받기- 이름(문자)
            String phone = rs.getString(3);

            System.out.printf("id: %s\t name: %s\t phone: %s \n",id, name,
phone);
        }

    } catch (SQLException e) {
        System.out.println(e);
    } finally { // 연결 끊기
        JdbcTemplate.close(rs);
        JdbcTemplate.close(stmt);
        JdbcTemplate.close(conn);
    }
}
```

2) where 조건 select문 출력

```

public static void selectTest03() {
    Connection conn=null;
    PreparedStatement pstmt=null;
    ResultSet rs= null;

    try {
        conn=JdbcTemplate.getConnection();
        pstmt=conn.prepareStatement("select id, name, phone from user where
id=?");

        //setInt(?어디, 값);
        //(int parameterIndex, int x);
        pstmt.setInt(1,1); //? 자리에 1 넣어줌

        rs=pstmt.executeQuery();

        while(rs.next()){ // 커서 이동
            int id= rs.getInt(1);
            String name= rs.getString("name");
            String phone = rs.getString("phone");

            System.out.printf("id: %s\t name: %s\t phone: %s \n",id, name,
phone);
        }

    } catch (SQLException e) {
        System.out.println(e);
    } finally { //연결 해제
        JdbcTemplate.close(rs);
        JdbcTemplate.close(pstmt);
        JdbcTemplate.close(conn);
    }

}

```

JDBC 실습 - INSERT/UPDATE/DELETE

SELECT문의 결과는 ResultSet으로 받아야 하지만, INSERT/UPDATE/DELETE (+DDL)의 경우에는 처리된 row의 행 수가 리턴된다. (ex: Query OK, 0 rows affected) 또 statement로 sql 실행시

`executeQuery()`가 아닌 `executeUpdate()`를 사용해야한다.

- + JDBC는 기본적으로 자동 커밋 설정이 true이므로 작업 성공/실패에 따라 커밋/롤백을 실행하려면 `.setAutoCommit(false)`로 연결의 자동 커밋을 비활성화하도록 하자!

```

con.setAutoCommit(false);

try {
    int row = pstmt.executeUpdate();

    if (row > 0) {
        con.commit();
    }
}

```

```

    } else {
        con.rollback();
    }
} catch (SQLException e) {
    con.rollback();
    throw e;
}

public static void main(String[] args){
    Connection conn = null;
    PreparedStatement pstmt=null;
    int row=0; //rs 대신 row 선언

    try{
        conn= getConnection();
        pstmt=conn.prepareStatement("insert into user (name, phone) values
(?,?)");

        pstmt.setString(1,"hong-gildong");
        pstmt.setString(2, "010-0000-0000");

        row=pstmt.executeUpdate(); // executeUpdate 사용

        if(row>0){ //하나 이상 insert 됐는지에 따라 커밋/롤백
            commit(conn);
        } else{
            rollback(conn);
        }
    }catch (SQLException e){
        System.out.println(e);
    } finally {
        close(pstmt);
        close(conn);
    }
}

```

JDBC 실습 - 프로시저 호출

CallableStatement, prepareCall로 호출, registerOutParameter()로 함수 반환값 등록, execute()로 실행

```

public static void main(String[] args){
    Connection conn =null;
    CallableStatement cstm =null;

    try {
        conn=getConnection();
        cstm = conn.prepareCall("call add10(?,?)");
        cstm.setInt(1,20);
        cstm.registerOutParameter(2, Types.INTEGER);
        // 반환값이 int
    }
}

```

```

        cstm.execute(); //sql 실행
        System.out.printf("call add10 결과: %d\n",cstm.getInt(2));

    } catch ( SQLException e){
        System.out.println(e);
    }finally {
        close(cstm);
        close(conn);
    }
}

```

<함수 호출> 반환값이 먼저 온다.

```

CallableStatement cstmt =
    con.prepareCall("{ ? = call calculate_salary(?) }");

cstmt.registerOutParameter(1, java.sql.Types.DECIMAL);
cstmt.setInt(2, empNo);
cstmt.execute();

```

JDBC 템플릿

연결, 자원해제, 트랜잭션 커밋/롤백 같은 반복 코드를 공통화

```

//1. 연결 생성
public static Connection getConnection() {
    Connection conn = null;
    try {
        String host = "localhost";
        String port= "3306";
        String database="jdbctest";
        String user ="root";
        String password="1234";

        String url =String.format("jdbc:mysql://%s:%s/%s?user=%s&password=%s",
            host, port, database, user, password);
        //System.out.println(url);

        conn = DriverManager.getConnection(url);
        conn.setAutoCommit(false);

    } catch (SQLException e) {
        System.out.println("[ERROR] connection error!");
        System.out.println(e);
    }
    return conn;
}

//2. 연결 해제
public static void close(Connection conn){

```

```

        if(conn != null){
            try{
                conn.close();
            } catch (SQLException e){
                System.out.println("[ERROR] connection close error!");
                System.out.println(e);
            }
            conn=null;
        }
    }

    public static void close(Statement stmt){
        if(stmt != null){
            try{
                stmt.close();
            } catch (SQLException e){
                System.out.println("[ERROR] statement close error!");
                System.out.println(e);
            }
            stmt=null;
        }
    }

    public static void close(ResultSet rs){
        if(rs != null){
            try{
                rs.close();
            } catch (SQLException e){
                System.out.println("[ERROR] resultset close error!");
                System.out.println(e);
            }
            rs=null;
        }
    }

    // 3. 저장
    public static void commit(Connection conn){
        try{
            conn.commit();
        } catch (SQLException e){
            System.out.println(e);
        }
    }

    // 4. 취소
    public static void rollback(Connection conn){
        try{
            conn.rollback();
        } catch (SQLException e){
            System.out.println(e);
        }
    }
}

```



- 어떤 sql문을 실행할지에 따라서 JDBC 명령어들이 조금씩 달라지니 주의하자.
- 맨 마지막에 항상 역순으로 Close 하는거 잊지말기!